LURELINE

Product architecture · UX specification · Evidence engine · Validation plan

Product thesis: An evidence-first cyber-deception investigation workbench. It turns a reproducible synthetic event stream into transparent alerts, a supported path, uncertainty markers and a measurable benchmark. It is not a chatbot, not a real honeypot and not a claim of live security monitoring.

Part I — Brutal red-team review

1. Product differentiation

A dark SOC dashboard, node graph and alert cards are commodity UI patterns. The defensible differentiator is not the theme: it is provenance on every edge, explicit uncertainty, a ground-truth benchmark and a baseline that may sometimes outperform the proposed method.

2. Problem alignment

Keep the official Cybersecurity & Blockchain track. Implement the central behaviors in the stated challenge: decoy assets, interaction detection, and reconstruction of movement through a simulated network. Do not drift into a general SIEM, chatbot or security analytics dashboard.

3. Technical validity

A graph line is not evidence. Each path edge must point to one or more event records and satisfy a defined graph/session rule. Do not use ground truth inside the algorithm being evaluated. Separate scenario fixtures, reconstruction logic and evaluation code.

4. Data honesty

Synthetic records cannot demonstrate real-world detection performance. Label every scenario and metric as synthetic; never say “live”, “real attacker”, “confirmed compromise” or “threat actor identified”. Do not fabricate precision/recall; compute them from explicit sets.

5. Scope risk

A real honeypot, backend, authentication, live traffic, packet capture, credentials, complex graph libraries and AI-generated analysis are out of scope for the first version. They increase failure risk without being necessary to demonstrate the core mechanism.

6. UI/UX risk

A graph alone is hard to evaluate. The user must be able to select an alert, edge or event and inspect its evidence. Every status must have a meaning; avoid decorative charts, random metrics, fake terminal animation and nonfunctional controls.

7. Architecture risk

Do not let UI components generate their own facts. Use a pure domain engine and one canonical scenario result; derive all cards, alerts, edges, timeline and benchmark from it. Keep the engine testable without rendering the UI.

8. Security and provenance

Use no real scanning, public honeypot, credential capture or external attack execution. The listed GitHub projects are references only. Check current LICENSE files and event reuse rules before reusing code; preserve notices and disclose reused components.

Part II — Product and UX blueprint

Target user and core job

Primary user: a judge or security analyst evaluating a single simulated incident. Job to be done: “Show me what the system observed, why it raised an alert, which path edges the evidence supports, and what remains unknown.” The interface should answer this without a chat prompt.

Information architecture

- Incident header: scenario name/ID, SIMULATION status, run/replay/reset controls, scenario description.
- Network canvas: six fictional nodes (two decoys, four ordinary), predefined topology, selected-path highlighting, node/edge legend, keyboard-accessible selection.
- Event ledger: timestamp, event ID, session, source → destination, event type, detection status, filter/sort controls.
- Evidence inspector: selected event/alert/edge, supporting event IDs, fired rule, provenance, confidence limitations and unresolved questions.
- Benchmark area: ground truth versus predicted edges, true positives, false positives, missed edges, precision/recall where defined, and naive baseline comparison.
- Investigation summary: short deterministic summary generated from computed values, with no LLM and no unsupported assertions.

Interaction and accessibility rules

- Every graph node, edge, alert and event row must be selectable; selection updates one shared evidence inspector.
- Use text plus colour/icon for statuses; never rely on colour alone.
- Keyboard focus must be visible. Buttons and rows must have accessible names; tables must remain readable on narrow screens.
- Provide empty, loading (only if real work is occurring), invalid-data and incomplete-evidence states.
- Reset must clear scenario events, selected entity, alerts, path, benchmark and filters consistently.
- Use animation only for selection/replay. No fake real-time data or fake terminal output.
- Display units and metric definitions. Show N/A rather than divide-by-zero or invented percentages.

Part III — Technical architecture

Recommended architecture: React + TypeScript + Vite, browser-only and static deployable to Vercel. No backend is needed for the MVP.

- **src/domain/types.ts —** Node, Edge, SimulatedEvent, ScenarioDefinition, DetectionRule, Alert, ReconstructionResult and BenchmarkResult types.
- **src/data/scenarios.ts —** Four fixed synthetic scenario fixtures and their separate ground-truth/test expectations.
- **src/engine/validateEvents.ts —** Required-field validation, node-reference validation and explicit duplicate identity policy.
- **src/engine/detect.ts —** Pure deterministic rules; returns alerts linked to event IDs and rule IDs.
- **src/engine/reconstruct.ts —** Builds candidate path edges from observed records, session identity and valid topology; reports unsupported/ambiguous edges rather than inventing them.
- **src/engine/baseline.ts —** Naive comparator that connects consecutive events within each session after stable sorting; labelled intentionally simplistic.
- **src/engine/evaluate.ts —** Compares predictions with ground truth only after reconstruction and computes TP/FP/FN, precision and recall with zero-denominator handling.
- **src/state/useInvestigation.ts —** Owns selected scenario, run result, filters and selected event/alert/edge; no duplicated per-panel facts.
- **src/components/ —** IncidentHeader, NetworkGraph, EventLedger, EvidenceInspector, BenchmarkPanel, ScenarioControls and SimulationNotice.
- **src/tests or available test setup —** Pure unit tests for detection, duplicate policy, ordering, session separation, reconstruction and metrics. Use the existing test framework if present; do not add a heavy test stack solely for appearance.

Core invariants

- Ground truth is available only to the evaluator and scenario test UI, never to detection or reconstruction.

- An alert must reference an event ID and rule ID that actually exist.
- A reconstructed edge must cite supporting event IDs and satisfy the allowed topology/session rules.
- Events from different sessions must never be linked merely by timestamp proximity.
- All UI metrics must be derived from the current run result; no separately hardcoded counts.
- Deduplication uses a documented stable identity key, not fuzzy text similarity.
- All event times are simulated and reproducible. Tie-breaking must be deterministic.
- Baseline and evidence-aware reconstruction receive the same event input.
- Unknown/incomplete evidence reduces conclusions; it must not increase certainty.

Part IV — FINAL MASTER PROMPT for Google AI Studio Build mode

Copy the full prompt below into a new Google AI Studio Build-mode app. Do not add optional features until the acceptance criteria pass.

You are a senior product engineer, frontend architect, security analyst, UI/UX designer and test engineer. Build LURELINE, an original ORCATHON 2026 web app aligned with the Cybersecurity & Blockchain problem: simulate interactions with decoy digital assets, detect them transparently, and reconstruct only the movement paths supported by event evidence.

PRODUCT PRINCIPLE

Lureline is an evidence-first investigation workbench, not a chatbot. The primary experience is scenario selection → deterministic replay → event detection → evidence inspection → path reconstruction → benchmark comparison. It must feel like a usable analyst tool, not a generic dashboard theme. All records are synthetic; show “SIMULATION LAB · Synthetic events only · No live network monitoring” persistently.

OPEN-SOURCE REFERENCE POLICY

Study these only as conceptual references; do not clone, merge or copy their code, UI assets, text or distinctive visual design:
- https://github.com/bastian-red/honeypot — event structure, deterministic alerts and simulator/test workflow.
- https://github.com/AKJenaX/DcoY — deception and investigation workflow.
- https://github.com/brezgis/honeypot-dashboard — session timeline and event-history presentation.
- https://github.com/sssafeman/trap-house — decoy network and incident visualization.

Do not claim these are integrated. Before any later code reuse, identify the exact source, current license, required notices and event rules. Prefer original implementation.

STACK AND LIMITS

Use React + TypeScript + Vite. Make a static client-side app suitable for Vercel. Use fixed deterministic synthetic scenarios. No backend, database, API key, real network listener, live honeypot, scanning, credential capture, external attack execution or LLM-generated security claims. Prefer native SVG for the network graph if practical. Keep dependencies small. No authentication, server, chatbot or complex infrastructure.

REQUIRED SCENARIOS

Create six fictional network nodes: two decoys and four ordinary assets, with predefined edges and fictional labels. Implement four repeatable scenarios:

A) Decoy interaction: a session interacts with a designated decoy and a transparent rule flags the exact event.

B) Normal activity: no decoy destination; no decoy-interaction alert should fire.

C) Incomplete evidence: a required event/relationship is missing; show a partial path and explicitly state what cannot be concluded.

D) Noisy/adversarial input: duplicate event, out-of-order timestamps and unrelated sessions. Apply the documented duplicate identity rule, stable ordering and strict session isolation.

Each scenario must define its event fixture and expected ground-truth edges separately. Ground truth is evaluation metadata only and must never be used by detection/reconstruction to create the answer.

DOMAIN MODEL

Define TypeScript types for NetworkNode, NetworkEdge, SimulatedEvent, DetectionRule, Alert, ScenarioDefinition, ReconstructionResult and BenchmarkResult.

Each event includes id, scenarioId, simulated timestamp, sessionId, sourceNodeId, destinationNodeId, eventType, concise description and provenance “synthetic”. Validate fields and node references. Deduplicate only using a documented stable identity key; never fuzzy-merge records. Sort by timestamp with a deterministic event-ID tie-breaker.

Keep domain logic pure and separate from UI. Use modules for scenario fixtures, validation/deduplication, detection, reconstruction, naive baseline and evaluation. Use one canonical run result to drive all components.

DETECTION

Flag an event if its destination node is explicitly configured as a decoy. Each alert must reference the exact event ID and rule ID and explain the rule in plain language. Group by session; never link unrelated sessions because they are close in time. Distinguish observed event records from inferred path edges. A path edge must be supported by event IDs and valid topology. If missing, conflicting or ambiguous evidence exists, show “Insufficient evidence to confirm this connection.” Never invent actor attribution, confidence percentages, compromise status or remediation success.

SIGNATURE FEATURE: EVIDENCE INTEGRITY & BENCHMARK

After the reconstruction has run, compare its predicted edge set with ground-truth edges. Compute true positives (predicted and expected), false positives (predicted but not expected) and false negatives/missed edges (expected but not predicted). Compute precision = TP/(TP+FP) and recall = TP/(TP+FN) only when the denominator is non-zero; otherwise show N/A and explain. Show metric

definitions.
Implement a deliberately simple baseline that connects consecutive events within each session after stable timestamp sorting, without the evidence/topology checks. Compare both methods using identical scenario inputs. Label it “naive baseline”. Do not tune data or hardcode results to make Lureline always win. If the baseline performs better on a scenario, display that result honestly. Ground truth must not leak into the reconstruction algorithm. Label all benchmark results synthetic and explicitly state they do not prove real-world detection accuracy.

UI/UX
Build a coherent analyst workbench:
- Incident header with scenario ID, scenario description, persistent simulation notice and run/replay/reset controls.
- Network graph with six nodes, legend, decoy markings and selectable nodes/edges.
- Event ledger with timestamp, ID, session, source→destination, event type, detection status, sort/filter controls.
- Evidence inspector that changes when a graph edge, alert or event is selected and shows the supporting event IDs, fired rule, provenance and limitations.
- Benchmark panel comparing predicted vs expected edges and baseline metrics.
- Deterministic investigation summary calculated from current result data; no LLM.
Use restrained dark-console styling, clear hierarchy, readable density, strong contrast and responsive behavior. No chat panel, excessive neon, fake terminal animation, random counters, decorative charts, fake live indicators or nonfunctional controls. Use status text plus icons, visible keyboard focus, accessible names and usable mobile tables. Include empty and incomplete-evidence states.

STATE AND DATA CONSISTENCY
All KPI cards, alerts, timeline rows, graph highlights, summaries and benchmark results must derive from one canonical scenario/run state. No hardcoded independent metrics. Run must create a reproducible result. Reset clears events, alerts, selections, filters, highlights and metrics. Selecting an event/alert/edge updates the same evidence inspector. No stale state or duplicate events after reruns.

TESTS AND ACCEPTANCE
Execute available checks; do not claim success unless actually run:
1. App renders without runtime errors.
2. Scenario A flags expected decoy event(s).
3. Scenario B has no decoy-interaction alert.
4. Scenario C does not invent missing edges.
5. Scenario D handles duplicates, timestamp ordering and session isolation.
6. Every alert links to a valid event and rule.
7. Every highlighted edge exposes supporting event IDs.
8. KPI counts match canonical data.
9. Ground truth is used only by evaluation after reconstruction.
10. Both methods use identical scenario inputs.
11. Reruns are deterministic and reset fully clears state.
12. All controls work with keyboard and mouse.
13. Available TypeScript/build checks succeed.
14. Production build needs no secrets or backend.
If a check cannot be run, report “Not verified”; do not fabricate test results. Fix errors before polishing.

DELIVERABLES
Create complete source code and README with local run/build and Vercel deployment instructions, architecture, event schema, detection rules, metric definitions, limitations and provenance. At completion list files changed, features implemented, exact tests/commands run and results, known defects/unverified checks, and GitHub/Vercel next steps. Priority: correct data model and scenarios; evidence-linked detection/reconstruction; benchmark and adversarial tests; usable analyst UX; then visual polish. Do not stop at a static UI.

Part V — Validation matrix

Test area	Pass condition
Scenario A	Expected decoy event flagged; alert points to exact event and rule; path edges cite evidence.
Scenario B	No decoy alerts; no false malicious label for normal activity.
Scenario C	Missing relationship remains unresolved; no fabricated edge; benchmark reports missed edge if in ground truth.
Scenario D	Duplicate policy applied; timestamps sorted deterministically; unrelated sessions remain isolated.
Metric math	TP/FP/FN sets correct; precision/recall N/A when denominator is zero.
Ground-truth isolation	Reconstruction code cannot read expected edges; evaluator reads them only after predictions.
State integrity	All UI values derived from one run result; rerun is deterministic; reset clears every derived state.
Deployment	Vite production build succeeds; public Vercel URL passes the key scenario checks.

Live demo sequence (60–90 seconds)

- Open Lureline and point out the simulation-only label.
- Run Scenario A; click the decoy alert and inspect the exact source event and rule.
- Click a reconstructed edge and reveal the supporting event IDs.
- Show predicted-vs-ground-truth metrics and compare with the naive baseline.
- Run Scenario B to show no decoy alert for normal activity.
- Run Scenario C or D to demonstrate uncertainty or robustness to noisy events.
- Close with the honest limitation: this validates the reconstruction logic on controlled synthetic scenarios, not real-world security performance.

Name, official track and reuse boundary

Lureline is a working name, not a verified unique name, trademark or available domain. Check event submissions and public search before final submission. Keep the official problem statement; do not change it solely because another team has a similar project. Differentiation is the evidence and benchmark mechanism.

Do not claim real-world attack detection. The app uses synthetic data. Do not represent conceptual reference repositories as integrations. Verify their current LICENSE files and official hackathon rules before using any source code; preserve required notices and disclose reused code.

Public references

- https://github.com/bastian-red/honeypot
- https://github.com/AKJenaX/DcoY
- https://github.com/brezgis/honeypot-dashboard
- https://github.com/sssafeman/trap-house
- https://ai.google.dev/gemini-api/docs/aistudio-build-mode

Verification boundary: this document is an implementation specification, not proof that the app has been built, tests have passed, the name is unique, or all licenses/event rules have been cleared. Verify current repository contents, license terms and the official problem statement before submission.